

Clase 15

REPASO UNIDAD II

CC1002 Introducción a la
Programación

Datos compuestos

- Estructura (**struct**):
 - Tipo de dato que permite encapsular un conjunto fijo de valores (de uno o más tipos)
 - Representado por atributos, que conformar un único valor compuesto
 - Se le denomina **dato compuesto**
- Módulo **estructura**: contiene la función **crear** para definir datos compuestos

CC1002-2 Semestre Primavera 2021

Datos compuestos

```
import estructura

# fraccion: numerador (int) denominador (int)
estructura.crear('fraccion', 'numerador denominador')

# sumaFracciones: fraccion fraccion -> fraccion
# devuelve la fraccion que resulta al sumar dos fracciones f1 y f2
# ejemplo: sumaFracciones(fraccion(1, 2), fraccion(3, 4))
# devuelve fraccion(10, 8)
def sumaFracciones(f1, f2):
    num = f1.numerador * f2.denominador + f1.denominador * f2.numerador
    den = f1.denominador * f2.denominador
    return fraccion(num, den)

# Tests
f12 = fraccion(1, 2)
f34 = fraccion(3, 4)
assert sumaFracciones(f12, f34) == fraccion(10, 8)
```

CC1002-2 Semestre Primavera 2021

Listas

- Definición de la estructura para listas

```
import estructura

# lista: valor (any = cualquier tipo) siguiente (lista)
estructura.crear('lista', 'valor siguiente')

# ejemplo con una lista de Strings
unaLista = lista('pedro', lista('juan', lista('diego', listaVacía)))
```

CC1002-2 Semestre Primavera 2021

Listas

```
# Modulo lista.py
# identificador para listas vacias
listaVacia = None

# cabeza: lista -> any
# devuelve la cabeza de una lista (un valor)
def cabeza(unalista):
    return unaLista.valor

# cola: lista -> lista
# devuelve la cola de una lista (una lista)
def cola(unalista):
    return unaLista.siguiente

# vacia: lista -> bool
# devuelve True si la lista esta vacia
def vacia(unalista):
    return unaLista == listaVacia
```

CC1002-2 Semestre Primavera 2021

Listas

- Plantilla de función para procesar listas:

```
# procesarLista: lista(...) -> ...
# descripcion de la funcion
# ejemplo de uso
def procesarLista(unalista):
    if vacia(unalista):
        ...
    else:
        ... cabeza(unalista) ...
        ... procesarLista(cola(unalista)) ...

# Tests
...
```

CC1002-2 Semestre Primavera 2021

Abstracción funcional

- Al proceso de combinar dos o más funciones en una sola más genérica se le denomina **abstracción funcional**
- La abstracción funcional es muy beneficiosa en programación
- Requiere agregar/modificar parámetros a las funciones para hacerlas genéricas

CC1002-2 Semestre Primavera 2021

Abstracción funcional

- Funciones abstractas para listas
 - Mapa
 - Filtro
 - Fold (reducir)
- Reciben como parámetro otra función

CC1002-2 Semestre Primavera 2021

Funciones anónimas

- Funciones “sin nombre”
- Pensadas como funciones que se utilizan una única vez
- Para definir funciones anónimas en Python se utiliza la instrucción **lambda**

```
lambda id1, id2, ...: expresion
```

CC1002-2 Semestre Primavera 2021

Funciones anónimas

- Ejemplos:

```
lambda x, y: x + y
```

```
lambda x: x < 5
```

- Si la expresión se utiliza más de una vez, debe definirse como función usando la receta de diseño

CC1002-2 Semestre Primavera 2021

Árboles binarios

- Estructura recursiva en donde hay dos atributos del mismo tipo definido
 - Usualmente `izq` y `der`
- Definición estructura AB (de árbol binario)

```
import estructura

# AB: valor(any) izq(AB) der(AB)
estructura.crear('AB', 'valor izq der')

# identificador para AB vacio
arbolVacio = None
```

CC1002 Introducción a la
Programación

Funciones útiles para AB

```
# obtenerValor: AB -> int
# devuelve el numero en la raiz del arbol (que debe ser no vacio)
def obtenerValor(raiz):
    assert not vacio(raiz)
    return raiz.valor

# ramaIzq: AB -> AB
# devuelve subarbol izquierdo del arbol (que debe ser no vacio)
def ramaIzq(raiz):
    assert not vacio(raiz)
    return raiz.izq

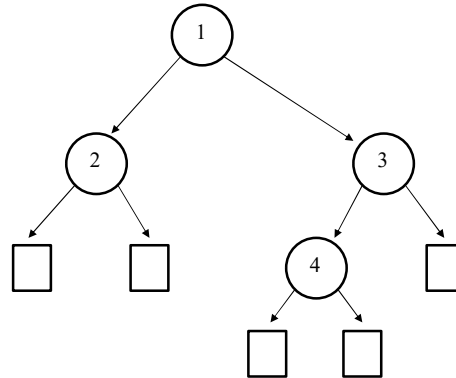
# ramaDer: AB -> AB
# devuelve subarbol derecho del arbol (que debe ser no vacio)
def ramaDer(raiz):
    assert not vacio(raiz)
    return raiz.der

# vacio: AB -> bool
# devuelve True si el arbol esta vacio
def vacio(raiz):
    return raiz == arbolVacio
```

CC1002 Introducción a la
Programación

Ejemplo de AB

```
nodo4 = AB(4, None, None)
nodo3 = AB(3, nodo4, None)
nodo2 = AB(2, None, None)
nodo1 = AB(1, nodo2, nodo3)
ab = nodo1
# ab es la raiz del arbol
# ab.valor es el valor de la raiz
# ab.izq es la rama izquierda
# ab.der es la rama derecha
```



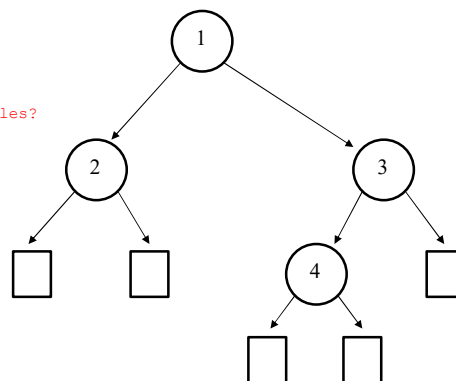
```
# En una sola linea de codigo
arbol = AB(1, AB(2, None, None), AB(3, AB(4, None, None), None))
```

CC1002 Introducción a la
Programación

Ejemplo de AB

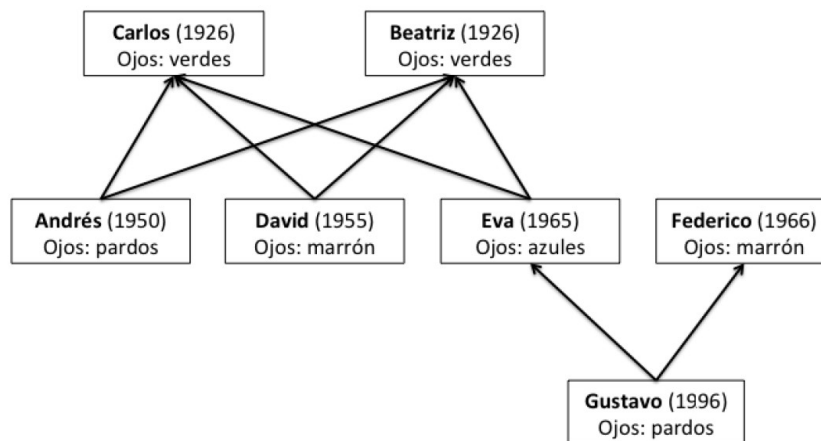
```
# Obtener el nodo que contiene el valor 4
nodo4 = arbol.der.izq

# A que corresponden las siguientes variables?
a = arbol.izq
b = arbol.izq.valor
c = arbol.der.izq.valor
d = arbol.izq.der
e = arbol.izq.der.valor
```



CC1002 Introducción a la
Programación

Ejemplo: árbol genealógico



CC1002 Introducción a la
Programación

Ejemplo: árbol genealógico

- ¿Cómo representar esta estructura?
 - Usaremos el concepto de **nodo**
 - Un nodoAG (nodo árbol genealógico):
 - Es vacío, o
 - Es un dato compuesto que tiene como atributos padre y madre (ambos nodoAG), nombre, nacimiento, y color de ojos

```
#nodoAG: padre(nodoAG) madre(nodoAG) nombre(str) nacimiento(int) ojos(str)
```

- Notar que hay dos atributos de tipo nodoAG

CC1002 Introducción a la
Programación

Ejemplo: árbol genealógico

- Módulo nodoAG:

```
import estructura

# nodoAG: padre(nodoAG) madre(nodoAG) nombre(str) nacimiento(int) ojos(str)
estructura.crear('nodoAG', 'padre madre nombre nacimiento ojos')

# identificador para nodos vacios
nodoAGVacio = None

# vacio: nodoAG -> bool
# devuelve True si el nodo esta vacio
def vacio(nodoAG):
    return nodoAG == nodoAGVacio
```

CC1002 Introducción a la
Programación

Ejemplo: árbol genealógico

- Creando el árbol genealógico:

```
# Primera generacion:
carlos = nodoAG(nodoAGVacio, nodoAGVacio, 'Carlos', 1926, 'verdes')
beatriz = nodoAG(nodoAGVacio, nodoAGVacio, 'Beatriz', 1926, 'verdes')

# Segunda generacion:
andres = nodoAG(carlos, beatriz, 'Andres', 1950, 'pardos')
david = nodoAG(carlos, beatriz, 'David', 1955, 'marron')
eva = nodoAG(carlos, beatriz, 'Eva', 1965, 'azules')
federico = nodoAG(nodoAGVacio, nodoAGVacio, 'Federico', 1966, 'marron')

# Tercera generacion:
gustavo = nodoAG(federico, eva, 'Gustavo', 1996, 'pardos')
```

CC1002 Introducción a la
Programación

Ejemplo: árbol genealógico

- Funciones sobre árboles genealógicos:

```
# Plantilla generica para una funcion que recibe un nodoAG
#
# unaFuncion: nodoAG -> ???
# def unaFuncion(unAG):
#     if vacio(unAG):
#         ...
#     else:
#         ... unaFuncion(unAG.padre) ...
#         ... unaFuncion(unAG.madre) ...
#         ... unAG.nombre ...
#         ... unAG.nacimiento ...
#         ... unAG.ojos ...
```

CC1002 Introducción a la
Programación

Ejemplo: árbol genealógico

- Función que determina si algún ancestro en la familia tiene ojos azules

```
# ancestroOjosAzules: nodoAG -> bool
# devuelve True si algun ancestro de unAG tiene ojos azules
def ancestroOjosAzules(unAG):
    ...

# Tests
assert not ancestroOjosAzules(carlos)
assert ancestroOjosAzules(gustavo)
```

CC1002 Introducción a la
Programación

Ejemplo: árbol genealógico

- Función `ancestroOjosAzules`
 - Si `unAG` es vacío, no tiene ancestro con ojos azules
 - Sino, si `unAG` tiene ojos azules, devuelve `True` (definición: uno es ancestro de sí mismo)
 - Sino, devolver si algún ancestro de `unAG` tiene ojos azules
 - Dos ancestros directos: padre y madre
 - Llamados recursivos usando ambos atributos

CC1002 Introducción a la
Programación

Ejemplo: árbol genealógico

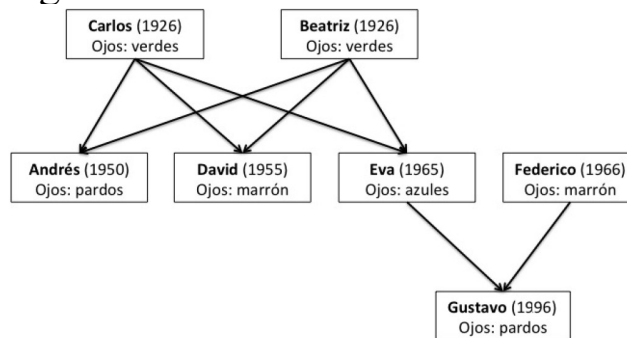
- Función `ancestroOjosAzules`

```
# ancestroOjosAzules: nodoAG -> bool
# determina si unAG tiene algun ancestro con color de ojos azul
def ancestroOjosAzules(unAG):
    if vacio(unAG):
        return False
    elif unAG.ojos == 'azules':
        return True
    else:
        return ancestroOjosAzules(unAG.padre) or ancestroOjosAzules(unAG.madre)
```

CC1002 Introducción a la
Programación

Para revisar en el apunte

- Revisar Sección 9.8 del apunte: árboles genealógicos descendentes



CC1002 Introducción a la
Programación

Para la próxima clase

- Leer los Capítulos 11.1 y 11.2 del apunte

CC1002 Introducción a la
Programación